

DOCKER EXAM COMMAND NOTES

Quick revision sheet based only on the two uploaded Docker practical manuals.

Focus: commands, syntax, what each option means, Dockerfile instructions, networking, volumes, and Docker Compose.

1. Installation & Verification

```
wsl --install
```

Windows: installs WSL2, which Docker Desktop can use as its backend.

```
docker --version
```

Checks the installed Docker version.

```
docker compose version
```

Checks the Docker Compose version.

```
docker run hello-world
```

Pulls the tiny hello-world image if needed, creates a container, and runs it to confirm Docker works.

2. Core Docker Commands — MUST KNOW

Command	Purpose
docker pull <image>	Download an image from Docker Hub.
docker images	List downloaded/local images.
docker run <image>	Create and run a container from an image.
docker run -it ubuntu	Run Ubuntu interactively and access its terminal.
docker ps	List currently running containers.
docker ps -a	List all containers, including stopped containers.
docker start <container>	Start an existing stopped container.
docker stop <container>	Stop a running container.
docker restart <container>	Restart a container.
docker rm <container>	Remove a container.
docker rmi <image>	Remove a Docker image.
docker logs <container>	View container logs; useful for troubleshooting.
docker exec -it <container> bash	Open an interactive Bash shell inside a running container.

3. docker run Options

Option	Meaning
-d	Detached mode — runs the container in the background.
-it	Interactive terminal mode; commonly used to enter a container shell.
-e	Set an environment variable.
--name	Give the container a custom/friendly name.
-p host:container	Bind/map a host port to a container port.
--network <name>	Connect the container to a specified Docker network.
-v host:container	Mount a host directory/path into a container.

Remember: -p is for ports, -e is for environment variables, -v is for volume/bind mounting, -d is background execution.

4. Image Tags / Versions

```
docker pull mysql:latest
docker pull mysql:8.2
```

Tags such as `latest` and `8.2` identify image versions/variants. Docker can share common image layers between versions.

5. MySQL Example

```
docker run --name test-mysql -e MYSQL_ROOT_PASSWORD=secret -d mysql
```

Creates a MySQL container named `test-mysql`, sets the root password through an environment variable, and runs it detached.

```
docker exec -it container_name bash
```

Enter the container terminal.

```
mysql -u root -p
```

Connect to the MySQL client as root and then enter the password.

```
docker run -d -e MYSQL_ROOT_PASSWORD=secret --name mysql_latest -p 8080:3306 mysql
```

Example of detached mode + environment variable + custom name + port binding.

6. Port Binding

Syntax: `-p <host_port>:<container_port>`

The container has its own isolated port. To access a service externally through the host, map a host port to the container port. Two containers cannot use the same host port at the same time.

```
docker run -d -e MYSQL_ROOT_PASSWORD=secret --name mysql_latest -p 8080:3306 mysql
```

Here `8080` is the host port and `3306` is the MySQL container port.

7. Docker Networks

```
docker network ls
```

List Docker networks.

```
docker network create <name>
```

Create a custom Docker network.

```
docker network create mongo-network
```

Containers on the same custom network can communicate internally without necessarily exposing their service ports externally.

Default networks: `bridge` = default/common network; `host` = shares the host network stack; `none` = no network access.

8. MongoDB + Mongo Express Example

```
docker run -d -p 27017:27017 --name mongo --network mongo-network -e
MONGO_INITDB_ROOT_USERNAME=admin -e MONGO_INITDB_ROOT_PASSWORD=qwerty mongo
```

MongoDB container: detached, port 27017 mapped, named `mongo`, attached to `mongo-network`, with root username/password environment variables.

```
docker run -d -p 8081:8081 --name mongo-express --network mongo-network -e
ME_CONFIG_MONGODB_SERVER=mongo -e ME_CONFIG_MONGODB_PORT=27017 -e
ME_CONFIG_MONGODB_ADMINUSERNAME=admin -e ME_CONFIG_MONGODB_ADMINPASSWORD=qwerty mongo-express
```

Mongo Express connects to the MongoDB container using the container name `mongo` on the shared network.

Mongo Express UI is accessed at `http://localhost:8081` according to the manual.

9. Dockerfile — Custom Image

A Dockerfile is a text file containing instructions Docker uses to build an image.

Instruction	Meaning
FROM python:3.11-slim	Choose the base image.

Instruction	Meaning
WORKDIR /app	Set the working directory inside the image/container.
COPY requirements.txt .	Copy the dependency file into the working directory.
RUN pip install --no-cache-dir -r requirements.txt	Install Python dependencies while building the image.
COPY . .	Copy the application files into the image.
EXPOSE 5000	Document the port the application listens on.
CMD ["python", "app.py"]	Default command run when the container starts.

10. Build & Run Your Own Image

```
mkdir docker-practical
cd docker-practical
docker build -t my-flask-app .
```

`<b>-t my-flask-app</b>` gives the image a tag/name. `<b>.</b>` means the current folder is the build context and contains the Dockerfile.

```
docker run -d -p 5000:5000 --name flask-container my-flask-app
```

Runs the custom Flask image in detached mode, maps host port 5000 to container port 5000, and names the container flask-container.

```
docker stop flask-container
docker rm flask-container
```

11. Volumes / Bind Mounts

Container filesystems are ephemeral. Volumes/bind mounts allow data to persist or be shared between host and container.

```
docker run -d -p 5000:5000 --name flask-container -v "%cd%":/app my-flask-app
```

Windows Command Prompt: mounts the current host folder into `<b>/app</b>` in the container.

```
docker run -d -p 5000:5000 --name flask-container -v "$(pwd)":/app my-flask-app
```

macOS Terminal: same idea, using the current directory.

```
docker restart flask-container
```

Restart the container after editing app.py; with the live mount, changes in the host folder are visible inside the container.

12. Docker Volumes Commands

```
docker volume ls
```

List Docker volumes.

```
docker volume create <name>
```

Create a named volume.

```
docker volume prune
```

Remove unused/dangling volumes.

Types: Named volumes are managed by Docker and explicitly named; anonymous volumes have no explicit name; bind mounts directly map host directories to container directories.

13. Docker Compose

Docker Compose defines and runs multiple related containers using one YAML file. The Compose file can define services, images, ports, environment variables, volumes, and networks.

```
docker compose up -d
```

Start the services defined in the Compose file in detached mode.

```
docker compose ps
```

Show the status of Compose services/containers.

```
docker compose down
```

Stop and remove the Compose containers and associated resources defined by the Compose application.

```
docker compose -f mongodb.yaml up -d
```

```
docker compose -f mongodb.yaml down
```

Use `<b>-f</b>` to specify a particular Compose YAML file.

14. Compose YAML Example to Remember

```
version: "3.9"

services:

web:
  build: .
  ports:
    - "5000:5000"
  volumes:
    - ./app
  redis:
  image: redis:alpine
  ports:
    - "6379:6379"
```

This defines a Flask web service built from the Dockerfile and a Redis service pulled from Docker Hub.

15. One-Page Last-Minute Revision

Pull image	<code>docker pull nginx</code>
Run background container	<code>docker run -d nginx</code>
Run with name	<code>docker run --name my-nginx nginx</code>
Map port	<code>docker run -d -p 8080:80 nginx</code>
Set env variable	<code>docker run -e KEY=value image</code>
List running	<code>docker ps</code>
List all	<code>docker ps -a</code>
Logs	<code>docker logs container</code>
Enter container	<code>docker exec -it container /bin/bash</code>
Stop	<code>docker stop container</code>
Start	<code>docker start container</code>
Restart	<code>docker restart container</code>
Remove container	<code>docker rm container</code>
List images	<code>docker images</code>
Remove image	<code>docker rmi image</code>
Create network	<code>docker network create name</code>
List networks	<code>docker network ls</code>
Build image	<code>docker build -t name .</code>
Run custom image	<code>docker run -d -p 5000:5000 --name app name</code>
Mount folder	<code>docker run -v host:container image</code>
List volumes	<code>docker volume ls</code>

Create volume	<code>docker volume create name</code>
Prune volumes	<code>docker volume prune</code>
Compose start	<code>docker compose up -d</code>
Compose status	<code>docker compose ps</code>
Compose stop/remove	<code>docker compose down</code>

16. Exam Traps / Remember These

- **Image vs Container:** Image = read-only template/blueprint; container = running or stopped instance of an image.
- **docker ps** shows running containers; **docker ps -a** also shows stopped ones.
- **docker stop** stops a container; **docker rm** removes a container.
- **docker rmi** removes an image.
- **-p** maps ports; format is host:container.
- **-e** sets environment variables.
- **-d** runs in the background.
- **--name** assigns a custom container name.
- **docker build -t name .**: -t names/tags the image; dot is the build context.
- **Dockerfile** must have no file extension; the manual specifically warns against Dockerfile.txt.
- **Compose** is for managing multiple related containers through a YAML configuration.
- **Volumes/bind mounts** prevent useful data from being tied only to a container's ephemeral filesystem.

Source note

These notes consolidate the commands and explanations appearing in the two uploaded practical manuals. The manuals cover Docker installation, core CLI commands, Dockerfiles, networking, volumes, and Docker Compose. ■filecite■turn0file0■L17-L28■
 ■filecite■turn0file1■L87-L104■